

FUNCIONES Y MODO DE USO API GATEWAY

Esta guía está dirigida a los desarrolladores de casino que necesiten interactuar con el proyecto ApiGateway para realizar transacciones, validar cuerpos de petición por medio de hash y unificar sus desarrollos en un único punto de acceso.

VirtualSoft
12/05/2026

Versión: 1.1



Tabla de contenido

1. Glosario	4
2. Requisitos Previos	4
3. Autenticación	4
4. Endpoints	4
4.1. Producción	4
4.2. Desarrollo	4
5. Archivo de Configuración: suppliersConfig.json	5
5.1. dataFormat	5
5.2. validateActionFor	6
5.2.1. Para buscar en url	6
5.2.2. Para buscar en un objeto o función	6
5.2.3. Para buscar en uno o más niveles de profundidad	6
5.3. action	7
5.3.1. extractPathSegments	7
5.3.2. cleanUrlSegments	8
5.3.3. methods	9
5.3.3.1. Configuración	9
5.3.4. compareValueProperties	10
5.3.4.1. Primer nivel	10
5.3.4.2. Varios niveles de profundidad	11
5.3.4.3. Mezclado	11
5.3.5. defaultAction	12
5.3.6. evaluateCondition	12
5.3.6.1. "operator": "type"	13
5.3.6.2. "operator": "equal"	13
5.4. Metodos	14
5.4.1. requireFullData	14
5.4.2. validateFor	14
5.4.3. construct	15

5.4.4.	parameters	16
5.4.4.1.	authenticate	16
5.4.4.2.	balance	16
5.4.4.3.	debit	16
5.4.4.4.	credit	17
5.4.4.5.	Rollback	17
5.4.4.6.	rollbackForTransactionId	17
5.4.4.7.	endRound	17
5.4.4.8.	debitAndCredit	18
5.4.5.	response	18
5.4.6.	continuesExecuting	20
5.4.6.1.	Ejecutar uno o más métodos de forma consecutiva	20
5.4.6.2.	Reutilizar un método definido	21
5.4.7.	isOnlyResponse	22
5.4.8.	validateUserAndPassword	23
5.4.9.	shouldFindUserInToken	23
6.	Funciones de personalización y acceso a datos	23
6.1.	Key	24
6.2.	Value	24
6.3.	Keys	25
6.4.	steps	26
6.5.	isArray	27
6.6.	freeSpin	27
6.7.	deepValue	28
6.8.	bonusGame	28
6.9.	extractValue	29
6.10.	customizeValue	30
6.11.	conditionalValue	30
6.12.	dateCustomization	31
6.13.	allowCustomization	31
6.14.	customizeDeepValues	32
6.15.	formatConfiguration	33
6.16.	mergeExtractedValueIntoRequest	34

7.	Motor para creación y verificación de hash	34
7.1.	Modo de firma raw	35
7.2.	Modo de firma RSA.....	38
7.3.	Reglas y limitaciones	38
8.	Decodificador cuerpo de peticiones	39
8.1.	Opciones de parametrización aes	40
9.	errorResponseConfig.json	40
9.1.	dataFormat	40
9.2.	códigos de error	40
9.3.	default.....	42
9.4.	structureErrorResponse	43
9.5.	responseHttpCodes	44
10.	Notas Adicionales	45
11.	Historial de versiones:	45

1. Glosario

- **requestAction:** Es la variable en donde se guarda el nombre final del método que se va a ejecutar enviado por el proveedor, este nombre corresponde a las claves definidas en la sección “**action**” del JSON, sino esta definido en el “**action**” se lanzará un error indicando que el método no esta definido.
- **methodName:** Es el nombre del método que se ejecuta, este método no corresponde a los métodos del proveedor, son los métodos del núcleo, los métodos disponibles son (*debit, credit, authenticate, balance, rollback, rollbackForTransactionId, endRound*), este nombre es el valor correspondiente a la clave “**requestAction**” de la sección “**action**” .
- **newMethod:** Es la variable que guarda el nuevo “**requestAction**” que se debe ejecutar según algunas configuraciones de la sección “**action**”.

2. Requisitos Previos

Esta documentación no es entregada a los proveedores ya que es para uso interno y solo el equipo de integraciones deberá tener acceso a la misma.

Por lo que los requisitos para el correcto funcionamiento serán los siguientes:

- Registrar **whitelist** en ambos lados, es decir, de lado del proveedor la dirección IP de nosotros y de lado de nosotros las direcciones IP del proveedor.
- El proveedor debe configurar el endpoint.

3. Autenticación

No tiene autenticación de acceso porque se tiene el sistema de login y tiempo de inactividad en la plataforma, si se hace una petición y el token ya fue vencido (*la sesión se cerró*) el API rechaza las peticiones devolviendo un error de sesión al proveedor

Las autenticaciones que hace esta API son las requeridas por el proveedor validando el cuerpo de las peticiones por medio de hash.

4. Endpoints

Para ambos casos se debe de cambiar el nombre del proveedor

4.1. Producción

Método	URL
POST	https://integrations.virtualsoft.tech/casino/genericTransactional/api/{{nombre-proveedor}}/

4.2. Desarrollo

Método	URL
POST	https://apidevintegrations.virtualsoft.tech/integrations/casino/genericTransactional/api/{{nombre-proveedor}}/

5. Archivo de Configuración: suppliersConfig.json

Cada configuración de cada proveedor debe ir dentro de una sección llamada con el mismo nombre del proveedor y debe ser escrito en mayúsculas.

Ejemplo:

```
{
  "BARBARABANG":{
  }
}
```

Dentro de la configuración del proveedor deben estar las siguientes secciones

```
"BARBARABANG": {
  "dataFormat": "json",
  "validateActionFor": {
    "type": "callable",
    "callable": "name"
  },
  "action": {
    "init": "authenticate",
    "bet": "debit",
    "win": "credit",
    "refund": "rollbackForTransactionId",
    "tournamentWin": "credit",
    "betAssociatedWin": "debit",
    "loyaltySystemPrizeWin": "credit"
  },
  "decoderConfiguration":{
    "steps":["bae64", "urlDecode", "gzip", "aes"],
    "options":{
      "passphrase":"secret passphrase",
      "cipher_algo":"AES-256-CBC",
      "flag":"OPENSSL_RAW_DATA",
      "iv":"secret iv (opcional)"
    }
  },
  "init": {
  }
}
```

5.1. dataFormat

Define que formato de datos usa el proveedor, el JSON solo recibe dos formatos (JSON, XML) por lo que también se debe considerar JSON a (*query_params* y *urlEncode*)

5.2. validateActionFor

Define desde donde se extraerá el nombre del método que se debe ejecutar, se puede configurar de 3 maneras:

5.2.1. Para buscar en url

```
"validateActionFor": {
  "type": "url",
  "callable": null
}
```

5.2.2. Para buscar en un objeto o función

“callable” define el nombre de la propiedad que guarda el nombre del método a ejecutar

```
"validateActionFor": {
  "type": "callable",
  "callable": "name"
}
```

5.2.3. Para buscar en uno o más niveles de profundidad

Esta configuración define dos al mismo tiempo, define la del paso anterior y adicional define un “deepCallable”.

Se debe establecer el valor por defecto, dentro de “deepValue” se define el camino para llegar al objeto que guarda el valor que se necesita, en este ejemplo funciona así:

transactions(objeto) – 0 (indica la posición de un array – actionType(objeto))

Por lo que la estructura final sería \$request->transactions[0]->actionType

Tener en cuenta: “deepCallable” solo se ejecuta si “callable” no es encontrado

```
"validateActionFor": {
  "type": "callable",
  "callable": {
    "callable": {
      "key": "actionType",
      "value": null
    },
    "deepCallable": {
      "value": null,
      "steps": {
        "deepValue": [
          "transactions",
          0,
          "actionType"
        ]
      }
    }
  }
}
```

```
}
}
```

5.3. action

Define la traducción de los métodos que se van a ejecutar y si es requerido define un ajuste adicional que cambia la forma en que se busca el nombre del método que debe ser ejecutado.

Los nombres que se definen como claves son los nombres que envía el proveedor y los valores serán los nombres de los métodos que se deben ejecutar

```
"action": {
    "authenticate": "authenticate",
    "balance": "balance",
    "bet": "debit",
    "win": "credit",
    "rollback": "rollback"
}
```

Tiene las siguientes configuraciones adicionales:

5.3.1. extractPathSegments

Extrae valores que llegan por la url de la petición pero que no son un queryParams valido, es un pathParams.

Define las posiciones desde el punto de inicio y define nombres para asignar a los valores extraídos, en el ejemplo esta sería la url de la petición:

Ejemplo Url de petición:

Metodo	URL
POST	rfranco/gamesessions/CASINO_VIRTUALSOFT/251P12941P22jx3wf74rwyqprtr6qm/balance

```
"balance": {
    "extractPathSegments": {
        "startFrom": "rfranco/",
        "positionSegmentsValue": [
            1,
            2
        ],
        "separator": "/",
        "nameSegments": [
            "casinoUid",
            "gameSessionUid"
        ]
    },
}
```

```

    "methodName": "balance"
  }

```

La configuración corta el path a partir del valor establecido en **“startFrom”** y crea un array separando por lo establecido en **“separator”**. En **“positionSegmentsValue”** se le está indicando que extraiga la posición 1 y 2 del array creado y que asigne los nombres que están en **“nameSegments”** a los valores extraídos (***Tener en cuenta que los nombres serán asignados de arriba hacia abajo al array, ejemplo casinoUid tendrá el valor de la posición 1).***

Finalmente lo que hace es fusionar los datos extraídos al request que envía el proveedor para poder acceder a esos datos desde el método que se va a ejecutar

“methodName” Indica el nombre del método que va a ser ejecutado, este nombre corresponde a los métodos propios y no los nombres de los métodos del proveedor

5.3.2. cleanUrlSegments

Decide que método debe ejecutar cuando llegan dos métodos con el mismo nombre.

Ejemplo:

Metodo	URLs
POST	rfranco/jackpot/wins rfranco/wins

En la configuración se debe poner siempre el nombre que llega por la URL

```

"wins": {
  "cleanUrlSegments": {
    "valueExpectedInArray": "jackpot",
    "newAction": "jackpotwins"
  },
  "methodName": "credit"
}

```

La configuración corta el path a partir del nombre del proveedor dejando únicamente el path con el nombre de los métodos y crea un array separando por (/). Toma el array resultante y busca si el valor esperado en **“valueExpectedInArray”** está en el array y si coincide cambia el **“requestAction”** por el valor establecido en **“newAction”**, sino indica que el método que debe ejecutar será el establecido en **“methodName”**.



5.3.3. methods

Cambia el método que se debe ejecutar según el valor enviado en una propiedad del request, se define en el action con el método que se envía por url y dentro de la configuración se extrae el valor de la propiedad del request

Metodo	URL
POST	api/amusnet/Deposit

5.3.3.1. Configuración

```
"Deposit": {
  "methods": {

    "Deposit": {
      "key": "Reason",
      "value": null
    },

    "ROUND_END": {
      "value": "ROUND_END"
    },

    "ROUND_CANCEL": {
      "value": "ROUND_CANCEL"
    },

    "JACKPOT_END": {
      "value": "JACKPOT_END"
    }
  }
}
```

Se debe definir la sección con el mismo nombre que llega en la URL, para este ejemplo la sección será **“Deposit”**, esta sección es la que extrae el nombre que llega por el request que será el método que se ejecuta, el valor de **“key”** corresponde al nombre exacto enviado por el proveedor, esta propiedad contiene el nombre del método que se debe ejecutar.

Ejemplo del request:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<DepositRequest>
  <UserName>Virtualsoft_STG@EGT</UserName>
  <Password>syjiaeWeQCtoXVCE</Password>
  <PlayerId>12941</PlayerId>
  <TransferId>25136413-d9c0-41be-a8cf-
ce939b776adas123</TransferId>
  <GameId>434</GameId>
```

```
<GameNumber>729060681660839348</GameNumber>

<SessionId>5e97435958bde1499541e33d631d1d59</SessionId>
  <Amount>700</Amount>
  <Currency>PEN</Currency>
  <Reason>ROUND_END</Reason>
  <PortalCode>DoradobetPE_STG_Portal_PEN</PortalCode>
  <PlatformType>DESKTOP</PlatformType>
  <BaseBet>700</BaseBet>
  <WinAmount>0</WinAmount>
</DepositRequest>
```

Note que el valor de “**Reason**” es **ROUND_END**, este valor se debe definir exactamente como lo envía el proveedor como una clave en la configuración y como valor de la clave “**value**”, se debe repetir para cada uno de los valores que envíe el proveedor, en este ejemplo hay 3 valores disponibles que pueden llegar por el mismo método **Deposit**.

Las opciones deben ser definidas como “**requestAction**” y ponerle el valor correspondiente al método que debe ser ejecutado.

Ejemplo:

```
"action": {
  "ROUND_END": "credit",
  "ROUND_CANCEL": "rollback",
  "JACKPOT_END": "debit"
}
```

5.3.4. compareValueProperties

Esta configuración tiene dos modos de uso, uno a primer nivel y otro a varios niveles de profundidad e incluso se podrían mezclar las dos.

5.3.4.1. Primer nivel

```
"credit": {
  "compareValueProperties": {
    "properties": {
      "undebited": true,
      "allow_multiple_transactions": true
    },
    "newMethod": "winBonus"
  }
}
```

La configuración define la sección de “**properties**” que es la que guarda todas las propiedades que deben ser comparadas del request, en este ejemplo son dos, **es importante que las claves definidas sean las mismas que las que llegan en el request**.

Los valores definidos funcionan como un “condicional” en donde si todos los valores de las propiedades son iguales a los definidos en la configuración se ejecutará el nuevo **requestAction** definido en **newMethod**.

5.3.4.2. Varios niveles de profundidad

```
"win": {
    "compareValueProperties": {
        "properties": {
            "type": {
                "steps": {
                    "deepValue": [
                        "bonus",
                        "type"
                    ]
                },
                "equalTo": "FREE_SPIN"
            }
        },
        "newMethod": "FREE_SPIN_WIN"
    },
    "methodName": "credit"
}
```

El concepto es igual al anterior pero aquí el valor que se desea comparar esta configurado como valor en “**equalTo**” y en este ejemplo solo se esta comparando una propiedad y es “**type**”, esta propiedad esta en el segundo nivel de un objeto por lo que debe ser extraída recorriendo el objeto completo.

Se define “**steps**” y dentro se define “**deepValue**”, deepValue debe ser definido como array [], dentro de las llaves en string y separado por comas se define el objeto padre y la(s) clave(s) que le proceden hasta llegar al nivel deseado, en este caso solo 1 – **type**.

Esto equivale a:

```
$request->bonus->type
```

Se el valor extraído es igual al definido en **equalTo** se ejecutará el método definido en **newMethod**, sino es igual entonces se ejecutara el método definido en **methodName**

5.3.4.3. Mezclado

```
"win": {
    "compareValueProperties": {
        "properties": {
            "type": {
                "steps": {
```

```

        "deepValue": [
            "bonus",
            "type"
        ],
        "equalTo": "FREE_SPIN"
    },
    "campaign": "freebet"
},
"newMethod": "FREE_SPIN_WIN"
},
"methodName": "credit"
}

```

El concepto sería el mismo solo en una propiedad se usa el **“deepValue”** para extraer el valor del request y en la segunda propiedad **“campaign”** no se debe configurar nada adicional, solo se debe poner el valor esperado que debería tener el request, las condiciones para cada caso son las mismas explicadas anteriormente.

5.3.5. defaultAction

Esta configuración define un requestAction por defecto en caso de que el proveedor tenga un método sin nombre o anonimo

Metodo	URL
POST	api/kagaming/

Configuración:

```

"action": {
    "defaultAction": "getBalance",
    "getBalance": "balance"
}

```

Esta configuración es mas sencilla, **“defaultAction”** define el **“requestAction”** que se debe ejecutar en caso de que el proveedor no envíe ningún método, este método por defecto debe ser definido en la sección **“action”** para poder indicar que método del núcleo se debe ejecutar.

5.3.6. evaluateCondition

Esta configuración indica que se debe validar una propiedad y según su valor o tipo se cambiará o no el **“requestAction”**

La configuración se puede definir de dos formas, cambiando el valor de **“operator”**

5.3.6.1. “operator”:”type”

Indica que la validación se hará verificando el tipo de variable y no su valor, los tipos disponibles para la validación son los siguientes y deben ir en el campo “**expected**”, en caso de coincidir se cambiará el “**requestAction**”

```
string, bool, int, float, array, object
{
  "action": {
    "win": {
      "evaluateCondition": {
        "property": "transactions",
        "operator": "type",
        "expected": "array",
        "newMethod": "deepWin"
      },
      "methodName": "credit"
    }
  }
}
```

5.3.6.2. “operator”:”equal”

Indica que se debe verificar el valor de la propiedad, en caso de coincidir se cambiará el “**requestAction**”

```
{
  "action": {
    "win": {
      "evaluateCondition": {
        "property": "typeRound",
        "operator": "equal",
        "expected": "free_round",
        "newMethod": "deepWin"
      },
      "methodName": "credit"
    }
  }
}
```

En ambos casos a excepción de “**operator**”, cada clave indica

- **property:** Es la propiedad que se debe validar del body de la petición
- **expected:** Si “**operator**” es “**type**” solo puede contener uno de los valores mencionados anteriormente, pero si “**operator**” es “**equal**” puede contener cualquier valor ya que este hace referencia al valor enviado por el proveedor
- **newMethod:** Define un nuevo “**requestAction**” que, si se cumple la función, sería el método que se ejecutaría
- **methodName:** Es el nombre del método que se debe ejecutar para el “**action win**” que es donde se encuentra la configuración, esto en caso de que la condición no se cumpla

5.4. Metodos

Esta sección no se llama como tal métodos, pero es en donde se definen todos los métodos del proveedor (**los definidos en la sección action y en newMethod**) y debe definirse con el mismo nombre que se puso en las claves de la sección “**action**”.

Ejemplo “action”:

```
"action": {
  "start": "authenticate",
  "balance": "balance",
  "play": "debit",
  "credit": "credit",
  "revoke": "rollback",
  "end": "end"
},
"start": {},
"balance": {},
"play": {},
"credit": {},
"revoke": {},
"end": {}
```

Dentro de cada método definido pueden definirse las siguientes secciones:

Nombre	Obligatoria
requireFullData	Si
validateFor	Si
construct	Si
parameters	Si
response	Si
continuesExecuting	No
isOnlyResponse	No
validateUserAndPassword	No
shouldFindUserInToken	No

5.4.1. requireFullData

Indica si el método que se va a ejecutar debe guardar información en transacciónApi, si se establece en true envía campos adicionales al núcleo.

Ejemplo:

```
"requireFullData": false
```

5.4.2. validateFor

Establece las validaciones que debe realizar el método, las opciones validas son (**userId**, **token**), se puede configurar una sola o las 2

Ejemplo:

```
"validateFor": ["token", "userId"]
```

5.4.3. **construct**

Funciona exactamente como un constructor en php, acá se pueden establecer valores adicionales que no pueden ser enviados en la sección de “**parameters**” para luego acceder a ellos desde la sección de “**response**”

Ejemplo:

```
"construct": {
  "token": {
    "key": "token"
  },
  "hashFromProvider": {
    "steps": {
      "deepValue": [
        "headers",
        "x-request-signature"
      ]
    }
  }
}
```

Acá las claves hijas de **construct**, es decir, **token** y **hashFromProvider** son claves esperadas por el núcleo y no pueden definirse de forma aleatoria, por lo que las claves hijas de **construct** son definidas por el modelo y no por el proveedor, los valores definidos en **key** serán los valores definidos por el proveedor.

Las claves que esta sección recibe ya están definidas, si se envía cualquier otra clave diferente al siguiente listado, el modelo no guardará el valor asignado

Claves recibidas por el modelo

```
'externalId',
'token',
'sessionId',
'userId',
'bonus',
'errorCode',
'message',
'signature',
'status',
'requestUUID',
'operatorId',
'requestId',
'provider',
'userName',
'password',
```

```
'hashFromProvider',
'originalTransactionId'
```

5.4.4. parameters

Define las variables que debe recibir cada método en el núcleo, las variables pueden ser definidas en desorden, pero deben estar bien nombradas y debe coincidir con la cantidad mínima de variables requeridas.

Hay una clase que define el orden y la cantidad de parámetros opciones y requeridos para cada método, se llama **structureParameters**

```
"parameters": {
  "transactionId": {
    "key": "fs_transaction_id"
  },
  "roundId": {
    "key": "related_fs_transaction_id"
  }
}
```

En este caso las claves hijas definidas en **parameters**, funcionan igual que en la sección **construct**, siendo las claves hijas definidas por el modelo (**las claves las puede ver en el siguiente apartado definidas por cada método disponible en el modelo**) y los valores definidos en **key** serían los definidos por el proveedor.

5.4.4.1. authenticate

```
'updateToken',          // mandatory - you can send it
{value:false} - bool (true|false)
'generatedHash',        // optional
'gameId',                // optional
'extractedValue'         // optional
```

5.4.4.2. balance

```
'currency',              // mandatory - you can send it
{value:null}
'generatedHash',         // optional
'gameId'                 // optional
```

5.4.4.3. debit

```
'gameId',                // mandatory
'transactionId',          // mandatory
'amount',                 // mandatory
'roundId',                // mandatory
'currency',               // optional - you can send it
{value:null}
'isFreeSpin',             // optional - bool (true|false)
'generatedHash',          // optional
'bets',                   // optional
'existTicketAllowed',     // optional
```

'allowChanIfIsEnd'	// optional
--------------------	-------------

5.4.4.4. credit

'gameId',	// mandatory
'transactionId',	// mandatory
'amount',	// mandatory
'roundId',	// mandatory
'currency',	// mandatory - you can send it
{value:null}	
'isEndRound',	// mandatory
'isFreeSpin',	// mandatory - you can send it
{value:false} - bool (true false)	
'generatedHash',	// optional
'isOnlyOneWin',	// optional
'allowChangIfIsEnd'	// optional

5.4.4.5. Rollback

'amount',	// mandatory - you can send it
it {value:0}	
"transactionId",	// mandatory
'roundId',	// mandatory
'gameId',	// mandatory - you can send it
it {value:null}	
'generatedHash',	// optional
'validationTicketValue',	// optional
"specificTransaction",	// optional
'allowChangIfIsEnd',	// optional
'validateValueOfTransaction',	// optional
'allowCreditTransaction',	// optional
'checkDeleteRound',	// optional
'status'	// optional

5.4.4.6. rollbackForTransactionId

'amount',	// mandatory - you can send it
{value:0}	
"transactionId",	// mandatory
'roundId',	// mandatory
'gameId',	// mandatory - you can send it
{value:null}	
'generatedHash',	// optional
'validationTicketValue',	// optional
"specificTransaction",	// optional
'allowChangIfIsEnd',	// optional
'validateValueOfTransaction',	// optional
'allowCreditTransaction',	// optional
'checkDeleteRound',	// optional
'status'	// optional

5.4.4.7. endRound

'roundId',	// mandatory
------------	--------------

```
'transactionId',          // mandatory
'gameId',                 // optional
'generatedHash'           // optional
```

5.4.4.8. debitAndCredit

```
'gameId',                // mandatory
'transactionId',         // mandatory
'debitAmount',           // mandatory
'creditAmount',          // mandatory
'roundId',               // mandatory
'currency',              // mandatory - you can send it
{value:null}
'isEndRound',            // mandatory - you can send it
{value:false} - bool (true|false)
'isFreeSpin',            // optional - bool(true|false)
'generatedHash',         // optional - you can send it
{value:null}
'allowChanIfIsEnd',      // optional
'isOnlyOneWin',          // optional
'existTicketAllowed',    // optional
'bets',                  // optional
```

5.4.5. response

Esta sección es la respuesta que el proveedor espera, se debe escribir la estructura tal cual la espera el proveedor, las claves serán las propiedades que el proveedor espera que le enviemos y los valores representan las variables que contienen los valores requeridos.

Ejemplo XML:

El proveedor espera

```
<?xml version="1.0"?> <AuthResponse>
  <Balance>0</Balance>
  <ErrorCode>1000</ErrorCode>

  <AuthenticationToken>354P12256081Phmnf9hquesz2bh2jtw</Authenticatio
nToken>
  <ErrorMes sage>OK</ErrorMessage>
</AuthResponse>
```

Asi se configura

```
"response": {
  "AuthResponse": {
    "Balance": {
      "key": "balance",
      "steps": {
```

```

        "formatConfiguration": {
            "type": "intval",
            "multiply": 100,
            "numberFormat": {
                "decimals": 0,
                "decimalSeparator": "",
                "thousandsSeparator": ""
            }
        },
        "ErrorCode": {
            "key": "errorCode"
        },
        "AuthenticationToken": {
            "key": "token"
        },
        "ErrorMessage": {
            "key": "message"
        }
    }
}

```

Ejemplo JSON:

El proveedor espera

```

{
    "playerId": "10052356",
    "sessionId": "cd73938622f963aa2391e1d94efb8d21",
    "currency": "USD",
    "balance": 1562,
    "status": "success",
    "statusCode": 0
}

```

Así se configura

```

"response": {
    "playerId": {
        "key": "userId"
    },
    "sessionId": {
        "key": "sessionId"
    },
}

```

```

    "currency": {
      "key": "currency"
    },
    "balance": {
      "key": "balance",
      "steps": {
        "formatConfiguration": {
          "type": "intval",
          "multiply": 100,
          "numberFormat": {
            "decimals": 2,
            "decimalSeparator": ".",
            "thousandsSeparator": ""
          }
        }
      }
    },
    "status": {
      "key": "message"
    },
    "statusCode": {
      "key": "errorCode"
    }
  }

```

En el caso del **response** el concepto de clave hija y **key**, cambian. Acá las claves hijas del **response** son las esperadas por el proveedor y los valores definidos en **key** son los definidos por el modelo, es al contrario que en **parámetros y construccy**.

También se puede cambiar el formato de respuesta únicamente para las respuestas.

Por ejemplo si el proveedor es JSON pero la respuesta la espera en formato XML se puede definir la propiedad **dataFormat: "xml"**.

Ejemplo:

```

"response": {
  "dataFormat": "xml"
}

```

Nota: Mas Adelante se explica el concepto de **"key, steps y formatConfiguration"**

5.4.6. continuesExecuting

Se puede usar para ejecutar uno o más métodos de forma consecutiva o para reutilizar un método definido.

5.4.6.1. Ejecutar uno o más métodos de forma consecutiva

```

"continuesExecuting": [

```

```
        "play",  
        "credit"  
    ]  
}
```

Las condiciones para usar esta configuración son:

- El proveedor puede enviar dos peticiones con nombres diferentes pero el body es igual o solo cambia una propiedad del request, en este caso se puede usar.

Ejemplo:

Para el ejemplo el “**credit**” representa ganancias otorgadas por el proveedor (*normales o bonos*) y el “**play**” representa una apuesta, entonces cuando se realiza una jugada normal llega el método **play** en url y realiza el **continuesExecuting** como se muestra en el ejemplo.

```
"play": {  
    "requireFullData": true,  
    "continuesExecuting": [  
        "play",  
        "credit"  
    ]  
}
```

Pero, si en url llega el método **credit**, indica que es un bono y se ejecutaría igual pero, el punto de inicio ya no es el **play** sino que sería el **credit**.

Ejemplo:

```
"credit": {  
    "requireFullData": true,  
    "continuesExecuting": [  
        "play",  
        "credit"  
    ]  
}
```

Se ejecuta de esta manera porque el proveedor envía solo una ganancia, para poder darse se necesita un debit, por lo que el debit queda en 0 y se da la ganancia.

5.4.6.2. Reutilizar un método definido

El proveedor puede enviar uno o más métodos de apuesta o ganancia con diferentes nombres y para no crear otra sección con el nuevo método se puede

reutilizar uno ya existente siempre y cuando sea compatible, es decir, las propiedades requeridas deben ser las mismas.

Ejemplo:

```
"WithdrawAndDeposit": {
  "methods": {

    "WithdrawAndDeposit": {
      "key": "Reason",
      "value": null
    },

    "ROUND_END": {
      "value": "WithdrawAndDeposit_ROUND_END2"
    },

    "ROUND_CANCEL": {
      "value": "WithdrawAndDeposit_ROUND_CANCEL"
    }
  }
}
```

Donde **WithdrawAndDeposit_ROUND_END2** cuando se define no define un nuevo método, sino que reutiliza los métodos ya existentes.

Ejemplo:

```
"WithdrawAndDeposit_ROUND_END2": {
  "continuesExecuting": [
    "WithdrawAndResponse",
    "ROUND_END2"
  ]
}
```

Los métodos definidos en **continuesExecuting** pueden ser ficticios y no necesariamente los que envía el proveedor, se pueden crear métodos nuevos para cumplir correctamente con la funcionalidad del proveedor sin necesidad de volver a crear otro método

5.4.7. isOnlyResponse

Usado cuando no es necesario procesar un método transaccional o llamar un método del GAME y solo se debe enviar una respuesta.

Ejemplo:

```
"CloseRound": {
  "isOnlyResponse": true,
  "response": {
    "Code": "0",
  }
}
```

```

        "Status": ""
    }
}

```

Se define marcando la propiedad **isOnlyResponse** en **true** y luego se define la sección **response**, en este caso no se usan las claves **key**, **value**, **steps**, solo se define la respuesta con los valores fijos que pide el proveedor

5.4.8. validateUserAndPassword

Indica si es necesario que el método valide usuario y contraseña con las guardadas en el modelo de secretos, tener en cuenta que si se marca en **true** se deben enviar en la sección de **construct** el usuario y la contraseña que envía el proveedor en el request.

Ejemplo Defición:

```

"Authenticate": {
    "requireFullData": false,
    "validateUserAndPassword": true
}

```

Ejemplo Construct:

```

"construct": {
    "userName": {
        "key": "UserName",
        "value": null
    },
    "password": {
        "key": "Password",
        "value": null
    }
}

```

5.4.9. shouldFindUserInToken

Indica si el método que se ejecuta debe extraer el id del usuario del token recibido

```

"win": {
    "requireFullData": true,
    "shouldFindUserInToken": true
}

```

Se puede usar cuando en el método **rollback** o **credit** el proveedor no envía el id del usuario y si o si se debe ejecutar el método, aunque el token ya este vencido

6. Funciones de personalización y acceso a datos

Algunas funciones también se pueden usar dentro de la sección del **action**, tener en cuenta que todas las claves a excepción de **key** y **value** solo se pueden usar dentro de **steps**.

6.1. Key

Es usado en las secciones, **construc**, **parameters** y **response**, lo que hace es indicar de que propiedad del request se va a extraer el valor para ser asignado a la clave definida en el json.

Ejemplo:

```
"userId": {
    "key": "playerid"
}
```

Esto quiere decir que el valor de la propiedad **playerid** será asignado a una propiedad llamada **userId**.

También se puede usar si el proveedor usa el mismo método para dos casos diferentes y algunas de sus propiedades cambia, con la siguiente configuración se le indica que debe buscar en cualquiera de las opciones definidas, sino encuentra ninguna lanzará error, aunque se haya configurado un valor por defecto.

Ejemplo:

```
"amount": {
    "key": [
        "winAmount",
        "amount"
    ]
}
```

6.2. Value

Tiene dos usos importantes:

- Establece un valor por defecto que será tomado en caso de que la propiedad de **key** no sea encontrada o no exista, o si se cumplen ciertas condiciones como la de **conditionalValue** o la de **keys**, **freeSpin**, **amount**, en el último caso se establecer el **value** en 0 para los casos de **freeSpin**, si se cumple la condición de **freeSpin** se tomara el **value** y no el **key**

```
"userName": {
    "key": "UserName",
    "value": null
}
```

- Para definir la configuración de creación del hash

```
"generatedHash": {
    "value": {
        "hashCreationConfig": {
            "extractSecret": {
                "key": "SECRET_KEY"
            }
        }
    }
}
```

```

    },
    "options": {
      "mode": "hmac",
      "useKeys": true,
      "sort": "none",
      "algorithm": "sha256",
      "stringToSignOrder": [
        "payload"
      ],
      "stringToSignSeparator": "",
      "signer": "raw"
    }
  }
}

```

6.3. Keys

Su propósito es únicamente para **freeSpin**, tiene dos formas de uso dependiendo de la propiedad en la que se ponga (**amount**, **isFreeSpin**).

Funciona buscando en el request la propiedad o las propiedades configuradas siempre y cuando sean un objeto con varios niveles de profundidad, una vez extraído el valor el modelo valida si ese valor es considerado **true**.

- **amount**: Se usa en conjunto con **value** en donde **value** debe ser establecido en **0**, para este caso **keys** estará dentro de otra configuración llamada "**freeSpin**", adicionalmente se debe definir "**bonusValue**" en **false**.

Por lo que si la condición es considerada **true**, **amount** quedaría con el valor de **value** ignorando la propiedad configurada en **key**

```

"amount": {
  "key": "amount",
  "value": 0,
  "steps": {
    "freeSpin": {
      "keys": [
        "playingFreePlay"
      ],
      "bonusValue": false
    }
  }
}

```

- **isFreeSpin**: En esta propiedad se configura únicamente el **keys** sin necesidad de ponerlo dentro de **freeSpin** y se debe poner el **value** en **false**, si el valor es considerado **true** la propiedad **isFreeSpin** quedará con valor de **true**, sino, quedará con el valor establecido en **value** que debe ser **false**

```
"isFreeSpin": {
    "keys": [
        "promoId"
    ],
    "value": false
}
```

6.4. steps

Es el acceso a las funciones especiales del modelo, adicional también permite que se ejecute más de una función tomando el resultado de una función anterior

Ejemplo:

```
"amount": {
    "steps": {
        "deepValue": [
            "jackpotWinDetails",
            0,
            "transferAmount"
        ],
        "formatConfiguration": {
            "type": null,
            "split": 1000,
            "numberFormat": {
                "decimals": 2,
                "decimalSeparator": ".",
                "thousandsSeparator": ""
            }
        }
    },
    "value": 0
}
```

En el ejemplo primero se extrae el valor de la apuesta y luego se le da formato para finalmente ser asignado a **amount**

Funciones que se pueden combinar

- deepValue
- conditionalValue
- formatConfiguration
- extractValue
- allowCustomization

Se puede usar combinada, pero en la propiedad **key** se debe configurar un valor ficticio para que tome el valor de la función anterior y no el de **key**

Funciones que se pueden mezclar sin necesidad de tomar el valor de la función anterior

- bonusGame
- freeSpin

Funciones que no se pueden mezclar con otras

- mergeExtractedValueIntoRequest
- dateCustomization
- isArray
- customizeValue
- customizeDeepValues
- keys

6.5. isArray

Convierte la propiedad en un array vacío

```
"bets":{
  "steps":{
    "isArray":true
  }
}
```

6.6. freeSpin

Configuración para el amount, reconoce si la petición enviada es un freeSpin y establece el amount en 0

```
"freeSpin": {
  "keys": [
    "freebetsId"
  ],
  "bonusValue": false
}
```

Cuando son diferentes niveles

```
"freeSpin": {
  "keys": [
    "additional_details",
    "bonus_id"
  ],
  "bonusValue": false
}
```

6.7. deepValue

Esta configuración ayuda a extraer un valor que este en varios niveles de profundidad sin importar si es un objeto puro, o un array de objetos, se debe definir el “camino” hasta la propiedad final del objeto.

Ejemplo:

- **Objeto puro:**

```
"isFreeSpin": {
    "value": false,
    "steps": {
        "deepValue": [
            "additional_details",
            "is_freespin"
        ]
    }
}
```

- **Array de objetos:** Acá el 0 indica que después de **transactions** debe a la posición 0 de un array y luego a **amount**, el equivalente en php sería este `$request->transactions[0]->amount`.

```
"steps": {
    "deepValue": [
        "transactions",
        0,
        "amount"
    ]
}
```

6.8. bonusGame

Configuración para el **amount**, reconoce si la petición pertenece a un bono propio del juego y establece el **amount** en 0, también se puede usar para freeSpin solo si el proveedor no tiene bonos dentro del juego o no deben ser procesados por nosotros, si el proveedor tiene freeSpin y tiene bonos que debemos procesar se deben usar en conjunto **freeSpin y bonusGame**

expectValue es el valor que debe tener la propiedad definida en **key** para que se cumpla la condición y el **amount** quede en 0

```
"amount": {
    "key": "betAmount",
    "value": 0,
    "steps": {
        "bonusGame": {
            "key": "freeGames",
            "bonusValue": false,

```

```

    "expectedValue": true
  }
}

```

6.9. extractValue

Sirve para extraer un valor de un string usando un separador personalizado, además se puede definir en que posición está el string separado en caso de que en producción y desarrollo sean diferentes, también permite realizar validaciones con el valor extraído (**actualmente solo valida que la moneda no sea diferente**)

```

"extractedValue": {
  "key": "PortalCode",
  "value": null,
  "steps": {
    "extractValue": {
      "separator": "_",
      "isDevelopment": {
        "position": 3
      },
      "isProduction": {
        "position": 2
      },
      "validationRequired": {
        "isRequired": true,
        "validationType": "someCurrency"
      }
    }
  }
}

```

- **key:** Propiedad del request que contiene el valor que se desea extraer
- **value:** Valor por defecto que se usará en caso de que no se encuentre la propiedad establecida en **key**
- **separator:** Símbolo por el que se debe separar el string, es personalizado por lo que puede establecer cualquiera incluso un espacio en blanco " "
- **isDevelopment:** Indica en donde esta la posición extraída para desarrollo
- **isProduction:** Indica en donde esta la posición extraída para producción
- **position:** Luego de hacer la separación el modelo crea un array con los valores del string sin contar los separadores lo que genera un array, esta variable indica que posición del array se debe usar para extraer el valor requerido del array resultante
- **validationRequired:**

- **isRequired:** Indica si es necesario realizar una validación con el valor extraído
- **validationType:** Se debe poner el nombre de la función que se desea validar, las función debe estar definida en la clave **validate.php namespace**
Backend\integrations\casino\genericTransactional\models

6.10. customizeValue

Sirve para concatenar variables o ponerle un texto a la variable final extraída del request

Nota: Esta configuración es reutilizada por **allowCustomization** y **customizeDeepValues**

```
"transactionId": {
    "key": "TransferId",
    "steps": {
        "customizeValue": {
            "variables": [
                "TransferId",
                "_CREDIT"
            ],
            "functionFormatIsRequired": {
                "value": false,
                "nameFunction": null
            }
        }
    }
}
```

- **key:** Variable extraída del request para personalizar, en esta configuración es opcional definir la **key**
- **variables:** Define las variables que se concatenaran, el orden en que se concatenan es el orden definido en este array, si el texto escrito no corresponde a una propiedad valida del request lo tomara como texto plano y lo concatenara de todas formas
- **functionFormatIsRequired:**
 - **value:** Indica si la variable debe ser formateada por una función adicional, los valores validos solo son **true / false**
 - **nameFunction:** Establece el nombre de la función que se debe ejecutar, la función que recibe este parámetro son todas las funciones nativas de php, puede usar (**intval, floatval, strval, strtoupper etc**)

6.11. conditionalValue

Establece un valor personalizado según una condición basada en valores

```
"isEndRound": {
    "key": "closeRound",
```

```

        "value": false,
        "steps": {
            "conditionalValue": {
                "checkAgainst": [
                    null,
                    true,
                    ""
                ],
                "valueIfMatch": true
            }
        }
    }

```

- **Key:** El valor extraído del request que se tomará de referencia para realizar la condición
- **checkAgainst:** Son los valores con los que se va a comparar el resultado de la propiedad establecida en **key**
- **valueIfMatch:** Es el valor que tomará la propiedad si coincide con alguno de los valores de **checkAgainst**, sino coincide tomará el valor establecido en **value**

6.12. dateCustomization

Establece un formato de fecha personalizado

```

"time": {
    "value": "",
    "steps": {
        "dateCustomization": {
            "format": "d-m-Y H:i:s",
            "timezone": "America/Bogota",
            "convertToTimestamp": false
        }
    }
}

```

- **format:** Formato base en que debe estar la fecha
- **timezone:** Zona horaria en la que se creara la fecha
- **convertToTimestamp:** Indica si la fecha debe ir en **timestamp** o no, recibe **true** / **false**

6.13. allowCustomization

Pone una condición para realizar un **customizeValue**

```

"roundId": {
    "key": "GameNumber",
    "steps": {
        "allowCustomization": {

```

```

        "key": "Reason",
        "expectedValues": [
            "JACKPOT_END"
        ],
        "customizeValue": {
            "variables": [
                "JP",
                "GameNumber"
            ],
            "functionFormatIsRequired": {
                "value": false,
                "nameFunction": null
            }
        }
    }
}

```

- **key:** Nombre de la propiedad que se debe validar y de se necesario personalizar
- **expectedValues:** Listado de valores esperados para la condición, si el valor de la propiedad definida en **key** coincide con alguno de los valores configurados en este listado se ejecuta el **customizeValue**, sino, toma el valor de la propiedad del **key**
- **customizeValue:** Es el mismo funcionamiento explicado anteriormen

6.14. customizeDeepValues

Permite realizar extracción de la propiedad del request a varios niveles de profundidad sin importar si es un objeto puro o un array de objetos y reutiliza el **customizeValue** para personalizar la variable extraída del request

```

"transactionId": {
    "key": "transaction_id",

    "steps": {
        "customizeDeepValues": {
            "properties": [
                "transaction_id"
            ],

            "variables": {
                "transaction_id": {
                    "path": [
                        "message",
                        "bet_info",
                        "transaction_id"
                    ]
                }
            }
        }
    },
}

```

```

        "customizeValue": {
          "variables": [
            "transaction_id",
            "_C"
          ],
          "functionFormatIsRequired": {
            "value": false,
            "nameFunction": null
          }
        }
      }
    }
  }
}

```

- **key:** Propiedad que debe ser extraída del request, poner el **key** en esta configuración es opcional
- **properties:** Define los nombres de las propiedades van a ser extraídas para luego fusionar su valor en el request pero a primer nivel
- **variables:** Se deben definir con el mismo nombre que se definió en la sección de **properties**, sino se define igual no se encontrará el **path** para extraer el valor de la variable requerida, se pueden definir varias variables incluso sino tienen más de un nivel de profundidad
Se toman las claves definidas en **properties** y los valores extraídos para crear nuevas propiedades en el request, las propiedades creadas quedarán con los nombres definidos en **properties**
 - **path:** Indica el camino o propiedades por la que se debe pasar para extraer el valor requerido dentro del objeto
- **customizeValue:** Es el mismo funcionamiento explicado anteriormente

6.15. formatConfiguration

Cambia el formato de un número o string numérico, es usado principalmente para las propiedades **amount** y **balance**, pero puede ser usado para formatear números a string o de string a numérico según sea requerido **customizeValue** también puede formatear números pero la diferencia entre los dos es que **formatConfiguration** permite multiplicar o dividir el número, permite tener en cuenta decimales y personalizar el separador de decimales y de miles (, .).

```

"balance": {
  "key": "balance",
  "steps": {
    "formatConfiguration": {
      "type": "floatval",

```

```

        "numberFormat": {
            "decimals": 2,
            "decimalSeparator": ".",
            "thousandsSeparator": ""
        }
    }
}

```

- **key:** Acá si es obligatorio definir **key**, ya que extraerá el valor del **amount** para pasarlo a la configuración
- **type:** Es el nombre de la función que se usará para formatear el valor, normalmente se usa (**intval**, **floatval**), pero puede recibir opciones como (**strval**) entre otras
- **numberFormat:**
 - **decimals:** Define la cantidad de decimales con las que va a quedar el numero formateado
 - **decimalSeparator:** Define el separador de decimales
 - **thousandsSeparator:** Define el separador de milésimas

6.16. mergeExtractedValueIntoRequest

Crea una propiedad con el mismo nombre que la propiedad definida en el JSON asignándole el valor extraído y agregando esa propiedad al request

```

"userId": {
    "key": "code",
    "steps": {
        "mergeExtractedValueIntoRequest": {
            "separator": "P",
            "position": 1
        }
    }
}

```

- **key:** Es obligatoria, indica el nombre de la propiedad del request que contiene el string al cual se le realizará la extracción
- **separator:** Es el indicador para separar por fragmentos el string lo cual dejará un array con los valores resultantes al cuál se debe acceder por posiciones
- **position:** Esta es la posición de la que se va a extraer el valor extraído del request

7. Motor para creación y verificación de hash

El motor cuenta con dos modos principales de firma, **raw** y **rsa**

7.1. Modo de firma raw

Este modo de firma cuenta con las siguientes opciones de parametrización y su configuración se define dentro de **value**, en la propiedad **generatedHash**

```
"generatedHash": {
  "value": {
    "hashCreationConfig": {
      "extractSecret": {
        "key": "secret_key"
      },
      "options": {
        "mode": "hmac | hash",
        "sort": "none | first | deep",
        "useKeys": "true | false",
        "separator": ":", ".", ",", "-", "_", | (any other character)",
        "algorithm": "sha256 | md5 | sha512 | etc",
        "excludedKeys": "PublicKey || [PublicKey, AnotherKey]",
        "secretHeader": "x-api-key",
        "format": "XML | QUERY_PARAMS | URL_ENCODED",
        "secret": "llenado automáticamente",
        "publicKey": "llenado automáticamente",
        "headers": [
          "x-signature",
          "x-secret",
          "x-timestamp"
        ],
        "headersSeparator": " , . - _ = + * & : ; | @ c $ # %
etc",
        "signableParts": [
          "QUERY_PARAMS",
          "RAWBODY",
          "URL_ENCODED"
        ],
        "partsDelimiter": " . | , - _ = + * $ @ & etc",
        "stringToSignOrder": [
          "payload",
          "headers",
          "secret"
        ],
        "stringToSignSeparator": " . | , - _ = + * $ @ & etc",
        "signer": "raw | rsa",
        "signatureRSA": "llenado automáticamente"
      }
    }
  }
}
```



- **extractSecret:** En conjunto con **secretHeader** estas dos propiedades serán las usadas como **salt** de la firma creada para verificar la petición, se usa una u otra, nunca las dos al tiempo
 - **key:** Indica el nombre de la propiedad desde la cuál se extraerá el secret del modelo de secretos
- **options:**
 - **mode:** Indica el modo de firma que se debe usar, **hmac o hash**, si el modo es **hash** puede enviar cualquier algoritmo de firma incluso el **md5**
 - **sort:** Indica si los parámetros del request deben ordenados, tiene 3 opciones disponibles (**none, first, deep**)
 - **useKeys:** Funciona en conjunto con **separator** e indica si se deben usar solo los valores del request, o claves y valor para realizar la firma
 - **separator:** Esta propiedad se usa solo si **useKeys** es false e indica el separador que debe usarse entre cada valor, también puede recibir un ""

Ejemplo de useKeys y separator

Se tiene este request

```
{
  "propiedad1":1821,
  "propiedad2":"hola"
}
```

Y se define **useKeys = false y separator = |**, el resultado sería

```
{1821|"hola"}
```

Si **useKeys = true**, el resultado sería exactamente igual request

- **algorithm:** Define el algoritmo que se debe usar para la creación del hash, normalmente suelen ser **sha256 o sha512**
- **excludedKeys:** Indica si se debe excluir una o más propiedades del proceso de firma.

Ejemplo:

excludedKeys = ["hash"]

Al request llega

```
{
  "propiedad1":1821,
  "propiedad2":"hola",
  "hash":"kshfdkhsadfdsaf79r328r1kdskghsvnvgnohégiyeifn"
}
```

Cuando se realice la firma el request será

```
{
  "propiedad1":1821,
  "propiedad2":"hola"
}
```

- **secretHeader:** Funciona como un **secret** pero en este caso no es extraído del modelo de secretos, sino que será extraído de los propios headers enviados por el proveedor
- **format:** Es importante que el valor definido en esta propiedad sea preciso para garantizar el correcto funcionamiento del motor, los valores recibidos son (**XML, QUERY_PARAMS, URL_ENCODED**), si es **JSON** no es necesario definirlo ya que es el formato usado por defecto
- **secret:** Esta propiedad es llenada automáticamente con el resultado de **extractKey**
- **headers:** Guarda el orden y los headers que deben ser usados para la creación de la firma, es importante tener en cuenta que en esta sección los header se deben definir siempre en minúsculas y con guiones medio.

Ejemplo:

El proveedor envía: **X_SIGNATURE, X_HASH_SIGNATURE**

En la propiedad se deben poner así: **x-signature, x-hash-signature**

```
"headers": [
    "x-signature",
    "x-hash-signature"
]
```

- **headersSeparator:** Indica el separador que debe tener cada header, funciona igual que **separator**, también se puede definir en ""
- **signableParts:** Indica que se deben usar diferentes partes del request para realizar la firma, las opciones disponibles son:
(**QUERY_PARAMS, RAWBODY, URL_ENCODED**)
Nota: Actualmente no hay una implementación disponible con esta función
- **partsDelimiter:** Indica el separador que tendrá cada parte del **signableParts**, funciona igual que **separator** y **headersSeparator**, solo se usa si **signableParts** fue definido y puede ser ""
- **stringToSignOrder:** Indica en que orden deben estar las fuentes de datos para la firma, teniendo en cuenta que todo quedaría en un solo string concatenado, las fuentes disponibles son (**payload, headers, secret**)
Nota: Si se define **headers** se debe definir la configuración de **headers** y **headersSeparator**
Para casos como **hmac** no se debe definir el **secret** porque ya es extraído y normalmente no se requiere dos veces el secret, en este caso solo se debe poner **payload** y **headers** en el orden que se necesite
- **stringToSignSeparator:** Indica el separador que van a tener las partes de **stringToSignOrder**, funciona igual que los anteriores **separator**
- **signer:** Indica si el modo si la firma se debe hacer **rsa** o **hash**, las opciones válidas son (**rsa, raw**)

- **Opciones obligatorias:** Estas son las opciones que siempre debe configurar para el funcionamiento básico del motor de hash
 - mode
 - useKeys
 - sort
 - algorithm
 - stringToSignOrder
 - stringToSignSeparator

7.2. Modo de firma RSA

También usa la mayoría de parámetros descritos anteriormente a excepción de **mode**, **secretHeader** y **secretKey** y agrega estos nuevos parámetros específicos para **RSA**

- **signatureRSA:** Es la firma con la que se debe comparar la firma construida
- **publicKey:** Es la llave con la que se debe crear la firma RSA

Ejemplo general de uso:

```
"generatedHash": {  
    "value": {  
        "hashCreationConfig": {  
            "extractSecret": {  
                "key": "SECRET_KEY"  
            },  
            "options": {  
                "mode": "hmac",  
                "useKeys": true,  
                "sort": "none",  
                "algorithm": "sha256",  
                "stringToSignOrder":["payload"],  
                "stringToSignSeparator":"",  
                "signer":"raw"  
            }  
        }  
    }  
}
```

7.3. Reglas y limitaciones

- **JSON:**
 - Puede tener en cuenta las claves de las propiedades o usar solo los valores con un separador personalizado o con ninguno si no es necesario
 - Puede ordenar alfabéticamente a varios niveles de profundidad
- **XML:**
 - No soporta separator y useKeys

- **QUERY_PARAMS:**
 - Puede tener en cuenta las claves de las propiedades o usar solo los valores con un separador personalizado o con ninguno si no es necesario
- **URL_ENCODED:**
 - No soporta separator y useKeys
- **Todos los formatos:**
 - Soporta el RAW puro sin ordenar ni alterar usando hmac o hash
 - Puede ordenar alfabeticamente
 - Puede excluir una o más claves del request
 - Puede usar una clave del modelo de secretos y ponerla antes o después del request
 - Es decir **{body xml} . \$clave secretos; o \$clave secretos . {body xml};** O usar una clave que venga por los headers
 - Soporta cualquier algoritmo (**md5, sha256 o sha512**)
 - Tiene dos modos de firma (hash o hmac)
 - Puede usar un valor enviado por headers para realizar la firma del hmac
 - Puede concatenar valores de header para crear la firma (**los header quedan ordenados segun se definan**)
 - Puede escoger los parametros especificos con los que se firmara el request (**el orden de los parametros será el definido en el parametro correspondiente - el modelo no lo organiza**)
 - Si se escogen los campos específicos a firmar también se puede usar solo los valores con un separador personalizado para crear la firma.

8. Decodificador cuerpo de peticiones

Puede decodificar uno o más algoritmos de forma simultanea si es necesario, las opciones de configuración son:

```
"decoderConfiguration":{
  "steps":["bae64", "urlDecode", "gzip", "aes"],
  "options":{
    "passphrase":"secret passphrase",
    "cipher_algo":"AES-256-CBC",
    "flag":"OPENSSL_RAW_DATA",
    "iv":"secret iv (opcional)"
  }
}
```

Actualmente soporta 4 algoritmos de decodificación ("**base64**", "**urlDecode**", "**gzip**", "**aes**")

Se puedes decodificar todos de forma simultánea de ser necesario, para los casos de ("**base64**", "**urlDecode**", "**gzip**") no requiere opciones adicionales, "**aes**" si requiere opciones adicionales

8.1. Opciones de parametrización aes

- **passphrase**: Algoritmo de cifrado utilizado
- **cipher_algo**: Clave secreta usada para decifrar
- **flag**: Indica como se viene la data, 0 si viene en base64 (por defecto) y OPENSSL_RAW_DATA si es un raw puro
- **iv**: Vector de inicialización, debe ser el mismo usado al cifrar

9. ErrorResponseConfig.json

Cada configuración de cada proveedor debe ir dentro de una sección llamada con el mismo nombre del proveedor y debe ser escrito en mayúsculas.

Ejemplo:

```
"AMUSNET": {  
}
```

Dentro de la configuración de cada proveedor deben estar presentes las siguientes configuraciones:

9.1. dataFormat

Indica el formato en que se le debe responder al proveedor

```
"AMUSNET": {  
    "dataFormat": "xml"  
}
```

9.2. códigos de error

Define el mensaje y código que debe ser enviado al proveedor según el código de excepción generado por nosotros.

Ejemplo:

```
"AMUSNET": {  
    "dataFormat": "xml",  
    "10011": {  
        "supplierCode": 3100,  
        "supplierMessage": "EXPIRED"  
    },  
    "10012": {
```

```

        "supplierCode": 3100,
        "supplierMessage": "EXPIRED"
    },
    "21": {
        "supplierCode": 3100,
        "supplierMessage": "EXPIRED"
    },
    "20001": {
        "supplierCode": 3100,
        "supplierMessage": "INSUFFICIENT_FUNDS"
    }
}

```

Donde **supplierCode** y **supplierMessage** funcionan como variables de almacenamiento para construir el error y tomar el mensaje y código correspondientes a la excepción

- **supplierCode:** Almacena el código que le debemos enviar al proveedor
- **supplierMessage:** Almacena el mensaje de error que le debemos enviar al proveedor

En los códigos de error también se define el **idepotence**

Dentro del código de error puede estar este campo definido que indica que ese código de error se procesara como un **idepotence**

Ejemplo:

```

    "10001": {
        "isHandledError": true,
        "addToRootName": "Response",
        "structureErrorResponse": {
            "Balance": {
                "key": "balance",
                "steps": {
                    "formatConfiguration": {
                        "type": "intval",
                        "multiply": 100,
                        "numberFormat": {
                            "decimals": 0,
                            "decimalSeparator": "",
                            "thousandsSeparator": ""
                        }
                    }
                }
            }
        }
    }
}

```

```

    },
    "CasinoTransferId": {
        "key": "gameTransactionLogId"
    },
    "ErrorCode": {
        "value": 1000
    },
    "ErrorMessage": {
        "value": "OK"
    }
}
}

```

- **isHandleError:** Indica que el error debe ser procesado como **idempotence**
- **addToRootName:** Indica que se agregará una propiedad padre para todo el cuerpo de la respuesta usando el nombre del método enviado por el proveedor más lo definido en esta propiedad

```

<?xml version="1.0"?>

<WithdrawResponse>

    <Balance>11316243</Balance>

    <CasinoTransferId>499119</CasinoTransferId>

    <ErrorCode>1000</ErrorCode>

    <ErrorMessage>OK</ErrorMessage>

</WithdrawResponse>

```

WithdrawResponse, no se definió en **structureErrorResponse** pero la propiedad indica que se debe agregar y le concatena el **Response** y **WithdrawResponse** en este caso es el método enviado por el proveedor

9.3. default

Funciona igual que los códigos de error, el código y mensaje definidos en esta sección serán tomados en caso de que no se cumpla ninguno de los códigos de error definidos

```

"default": {
    "supplierCode": 3000,
    "supplierMessage": "INTERNAL_SERVER_ERROR"
}

```

9.4. structureErrorResponse

Responsable de guardar la estructura de respuesta del proveedor, se debe definir exactamente como el proveedor la espera, se deben agregar las indentaciones ya que el modelo al extraer la respuesta mantiene la estructura definida, solo para los casos que son XML se debe definir la estructura como si fuera JSON pero teniendo en cuenta la indentación y propiedades de XML en caso de que lleve propiedades

Ejemplo XML:

```
"structureErrorResponse": {
    "Balance": {
        "key": "balance",
        "steps": {
            "formatConfiguration": {
                "type": "intval",
                "multiply": 100,
                "numberFormat": {
                    "decimals": 0,
                    "decimalSeparator": "",
                    "thousandsSeparator": ""
                }
            }
        }
    },
    "CasinoTransferId": {
        "key": "gameTransactionLogId"
    },
    "ErrorCode": {
        "value": 1000
    },
    "ErrorMessage": {
        "value": "OK"
    }
}
```

Como se le envía al proveedor

```
<?xml version="1.0"?>
<WithdrawResponse>
    <Balance>11315553</Balance>
    <CasinoTransferId>499119</CasinoTransferId>
    <ErrorCode>1000</ErrorCode>
    <ErrorMessage>OK</ErrorMessage>
</WithdrawResponse>
```

Ejemplo JSON:

```
"structureErrorResponse": {
  "balance": {
    "key": "balance",
    "steps": {
      "formatConfiguration": {
        "type": null,
        "multiply": 1000,
        "numberFormat": {
          "decimals": 0,
          "decimalSeparator": "",
          "thousandsSeparator": ""
        }
      }
    }
  }
}
```

Como le llega al proveedor

```
"balance":291.00
```

Nota: En esta sección se pueden usar las propiedades **key**, **value** y **steps** con todas sus funciones

9.5. responseHttpCodes

Indica por código de error si se debe cambiar o no el código HTTP.

Las claves de esta configuración corresponden a los códigos de error explicados anteriormente y los valores indican el código HTTP que se le debe ser asignado a ese código de error.

```
"responseHttpCodes": {
  "10005": 422,
  "21": 422,
  "10030": 401,
  "20003": 403,
  "10011": 401,
  "20001": 422,
  "59": 422,
  "30004": 500,
  "300135": 422,
  "300137": 422,
  "10021": 422,
  "30012": 401,
  "10001": 200,
  "300078": 200,
}
```

```
"300082": 400,  
"300043": 400  
}
```

10. Notas Adicionales

Siempre se le debe dar formato a los archivos JSON para mantener la misma estructura y evitar conflictos al realizar los PRs.

11. Historial de versiones:

Versión	Fecha	Descripción del cambio	Responsable(s)
1.0	17/04/2026	Documentación inicial	Juan Esteban Lopez Arevalo
1.1	12/05/2026	Incorporación de evaluateCondition	Juan Esteban Lopez Arevalo